

Interview Prep: Gothia Chat Concierge

30-Second Pitch

This is a local fullstack hotel concierge app for guest check-in, room booking, check-out, internal ticket handling, and AI concierge chat.

I dockerized the project, set up a local Postgres/Supabase-style environment, moved critical ticket writes directly to Postgres for reliability, and used OpenAI server-side to generate more natural guest-facing confirmations after tickets are safely saved.

The main focus has been local reproducibility, cleaner architecture, secure API key handling, and robust hotel guest flows.

What The Project Does

- ? Guest check-in.
- ? Guest room chat.
- ? Room booking flow with total price before card details.
- ? Guest check-out.
- ? Internal dashboard for hotel staff.
- ? Service ticket creation from chat.
- ? AI-written guest confirmations.
- ? Local Docker-based development environment.

High-Level Architecture

Browser

- > React / TanStack Start app
- > Server routes / server functions
- > Direct Postgres writes for critical local data
- > OpenAI API for natural language confirmations
- > Internal dashboard reads tickets from Postgres

Tech Stack

- ? React
- ? TypeScript
- ? TanStack Start
- ? TanStack Router

- ? TanStack Query
- ? Tailwind CSS
- ? Bun runtime
- ? Docker Compose
- ? Postgres
- ? OpenAI API
- ? Local Supabase-compatible setup

Important Architecture Decision

We started with a local Supabase/PostgREST-style setup. On the Mac environment, PostgREST was starting successfully but loading 0 Relations in its schema cache.

That meant the service was technically running, but it could not see or expose the database tables. Because ticket creation is a critical path, I changed the backend so critical writes go directly to Postgres instead of relying on PostgREST.

The more reliable local path is now:

Chat -> app server -> direct Postgres insert

instead of:

Chat -> app server -> Supabase JS -> PostgREST -> Postgres

Service Request Flow

When a guest writes something like:

2 handdukas

the backend does this:

1. Receives the chat request.
2. Detects that the message is a service request.
3. Saves the user message to chat history.
4. Creates a WORK_REQUEST row in Postgres.
5. If OPENAI_API_KEY exists, asks OpenAI to write a natural confirmation.
6. Saves the assistant confirmation.
7. Shows the ticket in the internal dashboard.

Flow:

Guest message

- > service request detector
- > guest_transactions insert
- > AI-written confirmation
- > internal dashboard

Why Not Let AI Create The Ticket Alone?

AI is not deterministic enough for critical system actions.

The safer pattern is:

Reliable backend action first.
AI wording second.

That means the database update is controlled by deterministic backend code, while AI is used only to make the guest response feel natural and helpful.

This is one of the strongest technical points in the project.

OpenAI API Handling

OpenAI is called server-side only.

The API key is stored in .env:

```
OPENAI_API_KEY=sk-...
```

It is not committed to GitHub and is never exposed in the browser.

Important distinction:

ChatGPT Plus is not the same thing as OpenAI Platform API access. The app needs an OpenAI Platform API key with billing/quota enabled.

Useful verification command:

```
docker compose exec app sh -lc 'echo ${#OPENAI_API_KEY}'
```

If it prints 0, the container does not have the key.

If it prints a number such as 164, the key is present inside the container.

Docker

The app is meant to run locally with Docker Compose.

Useful commands:

```
docker compose up --build -d
docker compose ps
docker compose logs -f app
docker compose down
```

The local app runs at:

`http://localhost:3000`

Guest room chat:

`http://localhost:3000/gastrum?room=1204`

Internal dashboard:

`http://localhost:3000/internal`

Clean Architecture Points

The project separates UI, business logic, server logic, and persistence.

Examples:

- ? Check-in validation: `src/lib/check-in`
- ? Booking pricing: `src/lib/booking`
- ? Service request detection: `src/lib/chat/service-request-detector.ts`
- ? Direct Postgres access: `src/lib/db/postgres.server.ts`
- ? Ticket storage: `src/lib/transactions.server.ts`
- ? Internal ticket reads: `src/lib/tickets.functions.ts`

The goal is that React components handle UI/state, while domain rules and persistence live in dedicated modules.

How We Know It Works

We verify the full path with logs and UI:

```
[ServiceRequestDetector] Detected work request
[Transactions] Saved guest transaction
[Chat API] Returning AI-written service confirmation
```

Then we verify that the internal dashboard shows the ticket.

Working test:

1. Open:

`http://localhost:3000/gastrum?room=1204`

2. Send:

`2 handdukar`

3. Open:

`http://localhost:3000/internal`

4. Confirm the ticket is visible.

Troubleshooting Story

The main issue was that Docker services were running, but the database layer was not actually usable through PostgREST.

The key clue was:

```
Schema cache loaded 0 Relations
```

This showed that PostgREST had connected to Postgres but could not see any tables.

Instead of continuing to fight the local PostgREST setup, I changed the critical paths to direct Postgres access. This made ticket creation and internal ticket reads more reliable locally.

Likely Interview Questions

What was the hardest part?

The hardest part was the local backend setup with Docker, Postgres, migrations, and PostgREST.

PostgREST could be running while still loading 0 Relations, so the system looked healthy at the container level but failed at the application level. I added logs to trace the exact path and then moved critical writes directly to Postgres to remove that fragile dependency.

Why Docker?

Docker gives the project a repeatable local environment across Windows and Mac.

Instead of requiring each developer to manually install and configure databases and services, Docker Compose starts the app and infrastructure together.

Why direct Postgres?

Direct Postgres is simpler and more reliable for the critical local ticket path.

PostgREST is still useful, but for a core action like creating a service ticket, I wanted fewer moving parts and clearer debugging.

Why use AI at all?

AI improves the guest experience by writing natural, service-minded confirmations.

But AI is not trusted as the only mechanism for system actions. The backend creates the ticket first, then AI writes the response.

How are secrets handled?

Secrets are stored in `.env`, not committed to GitHub.

The OpenAI key is read by the server container and is not exposed to the browser.

Is this production ready?

This is a local MVP/demo architecture.

Before production, I would add:

- ? Real authentication.
- ? Protected internal routes.
- ? Production-grade secrets management.
- ? CI/CD.
- ? Automated tests.
- ? Rate limiting.
- ? Monitoring and structured logs.
- ? Managed Postgres or hosted Supabase.

Strong Closing Statement

The most important design choice was separating reliability from AI.

The backend performs the database action deterministically first. Then AI writes the guest-facing response.

That gives us both reliability and a better user experience.